RNGD ▪ CONCEPT-TO-SCHEMATIC AI STUDIO

A better way to lay out a hotel floor, and the studio around it

Layout optimization compared with the reference, then the whole system: its parts, how they work, how they differ and what they achieved

Naimul Haque ▪ Semester project with RNGD ▪ October 2026

What I will show

Part 1 ▪ Layout optimization

- The job, and the reference optimizer
- What was wrong with its floor plans
- My updated optimizer, in plain terms
- Side-by-side results on the same sites
- Letting it choose the building shape too

Part 2 ▪ The whole studio

- How we use the BaaP idea
- The five parts of the studio
- How each part works, with screenshots
- How the parts differ
- Results, limits and next steps

PART 1

Layout optimization

The reference optimizer and my updated one, on the same sites

The job: fit a hotel's rooms onto a floor

What goes in

- A floor outline on several levels, with the stairs and elevators marked
- A list of rooms to place: here 72 rooms, 69 of them guest rooms

What comes out

- Where the corridor runs (the same on every level, so floors stack)
- Where each room sits, with its door on the corridor

What makes a plan good

- As many rooms placed as possible
- A window for every guest room
- Little corridor, safe exits, accessible rooms

Why a computer should search

There are far too many possible plans to try by hand. An optimizer tries hundreds of plans, scores each one, and keeps improving.

Three test sites

The prototype's three floor outlines (fp1, fp2, fp3), each with the same 72-room program.

The starting point: the reference optimizer

The prototype uses a **genetic algorithm** (Busra's work). In plain terms:

1. Start with many random plans.
2. Score them and keep the better ones.
3. Make new plans by *mixing* two good plans and making small random changes.
4. Repeat, here for 500 rounds.

In this version a plan is described by corridor pieces that can sit **anywhere** on the floor. Rooms are then attached wherever they touch a corridor.

I had the result files for the three sites, not the source code. I rebuilt each delivered plan and scored it with the same scoring I use for my own.

Reference results, three sites

171 of 216

rooms placed

24

guest rooms without a window

What I saw in its floor plans

Rooms without windows

Nothing tells the search where the outside wall is, so whether a room gets a window is luck.

Wasted floor area

Rooms are dropped in one at a time. The gaps left over are odd shapes nobody can use.

Mixing plans rarely helps

Half of one plan glued to half of another seldom makes a sensible building.

Slow, and different every run

My own first version also asked an AI model to judge plans during the search: about ten minutes, and never the same answer twice.

No idea of the limit

It reports what it placed, but cannot say whether all the rooms could fit at all.

Shape is fixed

The building outline and the stair positions are given. It cannot try a different shape or more floors.

These points come from the reference results, from my first attempt, and from the review meeting on 10/08.

Same site, same rooms: before and after



Left: reference optimizer

- 61 of 72 rooms
- 7 guest rooms with no window (red dashed outline)
- Corridor wanders across the plate

Right: updated optimizer

- 72 of 72 rooms
- Every guest room has a window (blue line = window wall)
- Two straight corridors

Site fp1, Level 2.

The key change: only build plans an architect would accept

Instead of letting corridors and rooms go anywhere, the updated optimizer follows four simple rules. The search then only chooses *between sensible plans*.

1 Straight corridors

Corridors are straight lines, one room deep in from the outside wall, with rooms on both sides.

2 Equal bays

Each side of a corridor is cut into equal slots, one standard room wide. Rooms take whole slots.

3 Windows first

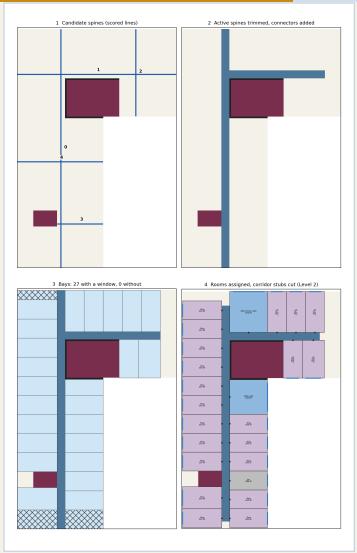
A guest room may only take slots that reach the outside wall. Room sizes are kept, not squeezed.

4 Floors stack

Every level has the same slots, so walls and plumbing line up from floor to floor.

The search now decides only two things: which corridors to use, and which rooms go first.

How one plan is built, in four steps



1. **Find the good corridor lines.** Every possible line is scored by how many window rooms it could serve. The best few become candidates (here five).
2. **Switch some on and connect them** to each other and to the stairs. Corridors are cut back so none runs past the last room.
3. **Cut the sides into slots.** Slots on the outside wall are made first, so window space is never wasted.
4. **Hand out the slots.** Important rooms choose first. Accessible rooms go near the elevator. A suite takes two slots.

On this site: 27 slots per floor, all with a window, and all 72 rooms placed.

How a plan is scored

Every plan gets one score between 0 and 1. It is plain arithmetic on the drawing, with **no AI model involved**, so the same plan always gets the same score.

What earns points

- Rooms placed (important rooms count more)
- Guest rooms with a window
- Little floor area spent on corridor
- Short walk to an exit, no long dead ends
- Enough accessible (ADA) rooms, on several floors

Also counted

- Service and accessible rooms close to the elevator
- Rooms that belong together are near each other
- Suites on corners
- Guest rooms spread evenly over the floors
- Rooms sharing walls (good for prefabrication)

The exit distance and dead-end limits are read from RNGD's own building-code rule file. A plan whose corridor misses an exit stair loses half its score.

Two ways to search, same plan builder

Updated genetic algorithm

The same idea as the reference, but on the new plan builder:

- starts from sensible corridor choices, not random ones
- mixes and tweaks plans each round
- polishes the best plan a little every round

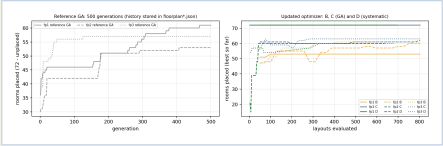
Good at: squeezing out the last bit of score.

Step-by-step search (now the default)

No mixing of plans at all:

- first tries every sensible combination of corridors
- then improves the best ones, one small change at a time, keeping a change only if the score goes up

Good at: being simple to explain, and giving exactly the same answer every time.



Results on the three sites

	Rooms of 216	No window	Score (0–1)	Time
Reference optimizer	171	24	0.863	n/a
My first attempt	174	14	0.889	336 s
Updated genetic alg.	**197**	**0**	**0.928**	72 s
Step-by-step search	**192**	**0**	**0.922**	59 s

- Same room list and the same scoring for every row.
- **The new plan builder gives the gain.** My first attempt had a better search but the old way of building plans, and kept the windowless rooms.
- The updated optimizers pass all six code checks on every site.
- The step-by-step search returned the identical plan when run twice.

Compared with the reference

+26 rooms

with the updated genetic algorithm

24 → 0

guest rooms without a window

Times are for all three sites together, on a laptop. The reference's run time was not recorded (n/a).

A finding: does mixing two plans helps?

The review meeting questioned whether mixing two plans makes sense. I recorded all 540 mixes in one run and compared each new plan with the two plans it came from.

8%
better than both parents

23%
same as the better parent

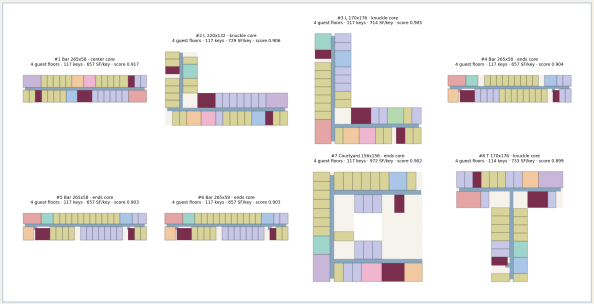
41%
in between the two

28%
worse than both parents

What this means

Mixing helps only now and then. That is why the default search does not use it, and why it is easier to explain: every improvement is one named change, for example “corridor 1 shortened at one end”. The genetic algorithm stays available as an option, because its rare lucky mixes still give it a slightly higher score.

Going further: let it choose the building shape too



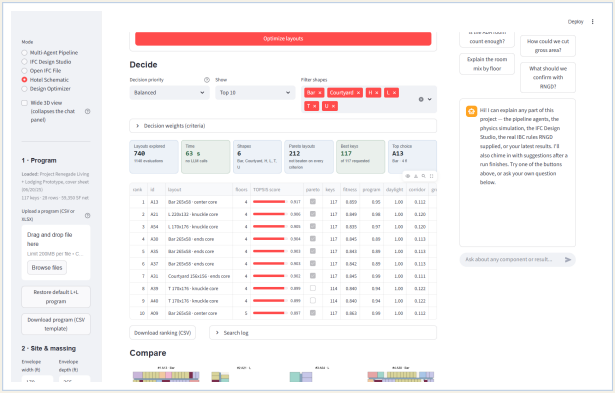
In the Hotel Schematic mode, only the room list and the buildable area are given. The optimizer then generates:

- **the shape**: bar, L, U, T, H or courtyard
- **where the elevator and stairs go**
- **how many floors**, adding a floor if the keys do not fit

For the 117-key Living + Lodging program

740 different layouts in about 36 seconds, with no AI model calls, and the same list on a repeat run.

Choosing among the layouts

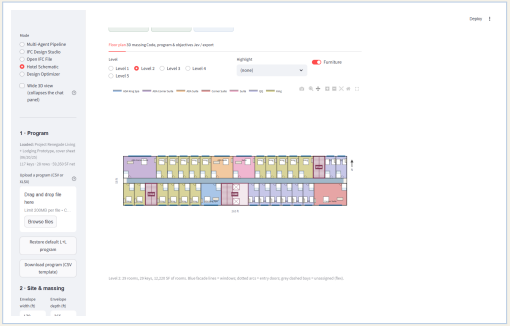


Which layout is best depends on what the owner wants, so that choice is kept separate from the search.

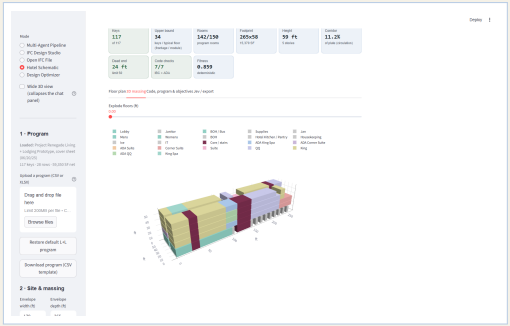
- Pick a priority: balanced, most keys, lowest cost, guest experience or low-rise
- The layouts are re-ranked instantly
- Layouts that another one beats on every point are flagged
- A drop-down shows the top 5, 10, 25 or all

First choice (balanced): a 265 × 58 ft bar with a central core, four guest floors, all 117 keys, 7 of 7 code checks.

Looking at one layout



Furnished floor plan (hover shows room data)



Stacked 3D view, coloured by room type

PART 2

The whole studio

The BaaP idea, the five parts, how they work and how they differ

How we use the BaaP concept

Building-as-a-Product (BaaP): design from standard, repeatable, prefabricated parts instead of one-off pieces. The studio applies the idea in three places.

In the layout optimizer

- The floor is a grid of equal bays, one standard room wide
- Rooms take whole bays and keep their size
- Same bays on every floor, so walls and plumbing stack

In the hotel generator

- Each room type is designed once and repeated
- One fixed room depth (26 ft)
- Rooms mirrored so bathrooms pair up; elevations show panels

In the agent pipeline

- One agent matches the customer's program to catalogue modules
- Python counts the modules needed
- Another agent flags conflicts with stacking rules and the site

Still open: the module catalogue in the pipeline is sample data. RNGD's real module and panel catalogue can replace it without changing the design of the system.

The studio at a glance



One web app with five modes. They share one knowledge base and one chat assistant. The optimizer of Part 1 runs as mode 5 and inside mode 4.

One rule across the whole system

The AI model reads and judges. Ordinary code does every calculation.

Done by code

Areas, counts, distances, site layout, floor plans, code checks, drawings, the optimizer.

Cannot make things up. Same input, same output.

Done by the AI, with guard rails

Reading a free-text request, finding limits in code documents, judging conflicts, the final recommendation.

Fixed answer formats, sources cited, "say unknown rather than guess".

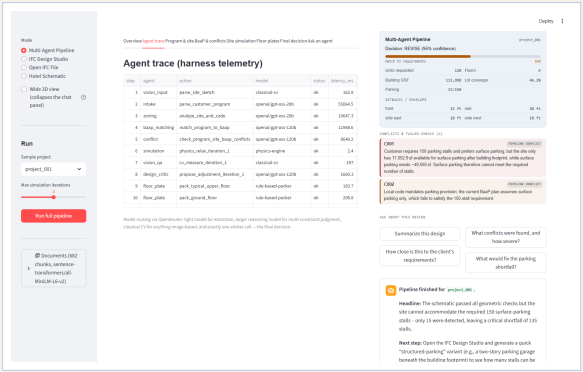
Free text from the AI

Explanations, report wording and chat answers.

Can be wrong. These are explanations; the checks computed by code are the facts.

Why it matters: a wrong number in a building design is expensive. Keeping the arithmetic out of the AI model is what kept invented numbers out of the results.

Part 1 of the studio: the multi-agent pipeline

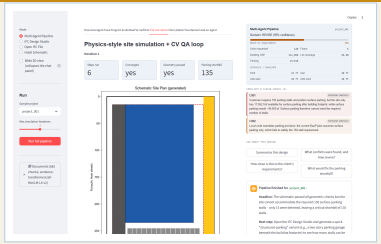


What it does: takes a customer's written request and a site, and decides whether the project is feasible.

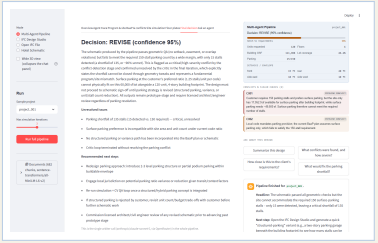
How: a chain of small AI agents, each with one job: read the request, look up the zoning limits, match rooms to BaaP modules, find conflicts, and give a final decision with reasons.

Every step is logged with what it did and how long it took.

The pipeline: site layout, checking and the decision



The building and parking are pushed into place like objects that may not overlap. The finished drawing is then measured again from the image, as an independent check.

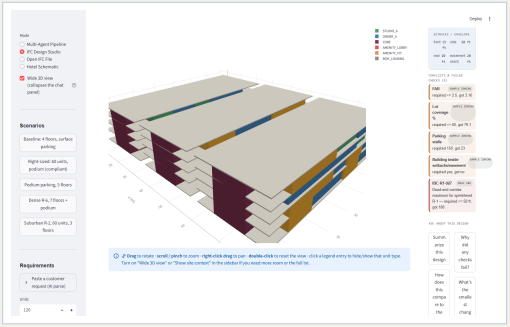


The final decision, with unresolved issues and next steps.

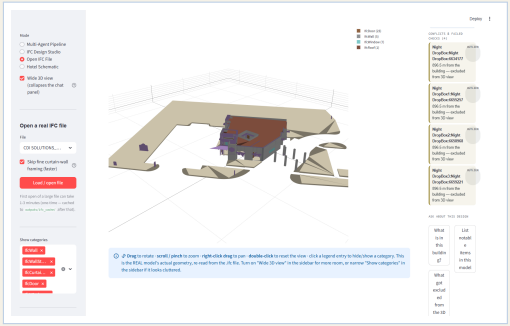
Result

Three full runs of the same project all gave the same decision (“revise”, because the surface parking cannot fit). The request was read correctly in 3 of 3 runs and all 9 zoning limits were found in 2 of 2 runs. The wording of the explanation varied between runs.

Parts 2 and 3: building models (IFC)

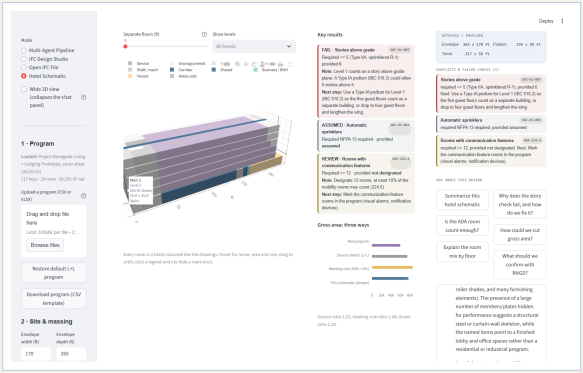


IFC Design Studio. Move a few sliders and get a real 3D building model that other design software can open, checked against zoning and RNGD's building-code rules. No AI model is used to create it.



Open IFC File. Load an existing building model (here a real 89 MB bank building), look at it in 3D, browse its elements and ask questions about it.

Part 4: Hotel Schematic



What it does: turns a hotel room program into a complete schematic design.

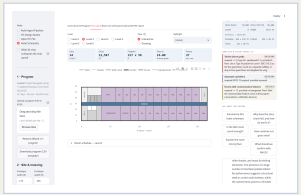
Default input: RNGD's Living + Lodging program, 117 keys in 28 rows. It can be edited or uploaded.

How: fixed rules, no AI model:

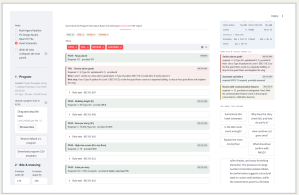
- guest rooms along a corridor with rooms on both sides
- public and service rooms on Level 1
- stairs at both ends, elevators at the core

Result: four different programs gave four different, correct buildings, and each rebuild was identical.

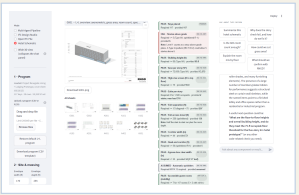
Hotel Schematic: plans, code check and drawings



Interactive floor plan of a guest floor



Check against RNGD's building-code rules and the ADA guest-room tables



A drawing set in RNGD's sheet format: plans, elevations, unit plans, 3D

A useful catch

With five guest floors the default program has six levels, one more than the five-story rule in RNGD's rule file allows. The check flags it and suggests four guest floors, which pass.

How the five parts differ

Part	Starts from	Method	AI model?	Repeatable?
1 Multi-agent pipeline	written request + site	AI agents, document search, site simulation	yes, to read and judge	decision yes, wording no
2 IFC Design Studio	a few sliders	rules that generate a 3D model	no	yes
3 Open IFC File	an existing model file	reading and displaying the model	only for chat	yes
4 Hotel Schematic	a hotel room program	rules for layout, code check and drawings	no	yes
5 Design Optimizer	floor outline + room list	search over many plans, scored by arithmetic	no (only for report text)	yes

- Parts 2 and 4 **build one design** from rules. Part 5 **searches many designs** and ranks them. Part 1 **decides** whether a request is feasible.
- Only part 1 depends on the AI model for its result, and its numbers still come from code.

Results in one slide

Layout optimization

- 171 → 197 rooms placed over three sites
- 24 → 0 guest rooms without a window
- All code checks pass on every site
- About 20 seconds per site, repeatable
- 740 hotel layouts with generated shapes in about 36 seconds

Quality

All 51 automated tests pass, including tests that drive the real app.

The rest of the studio

- Pipeline: same decision in 3 of 3 runs; every number in its explanation traced back to computed data
- Hotel generator: four programs, four correct buildings, identical on rebuild
- A complete drawing set in RNGD's sheet format, generated from the program
- RNGD's real building-code rules and the ADA standard are used for the checks

What is still open, and next steps

Honest limits

- The optimized hotel layout places all 117 keys but leaves out 8 of 150 rooms (retail, gym and some service rooms): the ground floor uses the same outline as the guest floors
- The reference optimizer's run time was not recorded, so I cannot compare speed with it
- Optimized layouts are shown as plans and 3D, not yet as drawing sheets
- Pipeline inputs and the BaaP catalogue are sample data
- No parking, structure or MEP yet

Next steps

- A wider ground floor (podium) so all public rooms fit
- Drawing sheets straight from the optimized layouts
- RNGD's real module and panel catalogue
- More of the building-code rule set
- Parking and structural grid

Questions for RNGD

Is a podium intended, as in the Living + Lodging prototype? Which module and panel sizes should the bays follow?

Thank you

In one sentence: once the optimizer can only build plans that make architectural sense, a simple, repeatable search places more rooms and gives every guest room a window.

Questions?

Details: walkthrough report, the paper, and the optimization-algorithms report in `reports/`